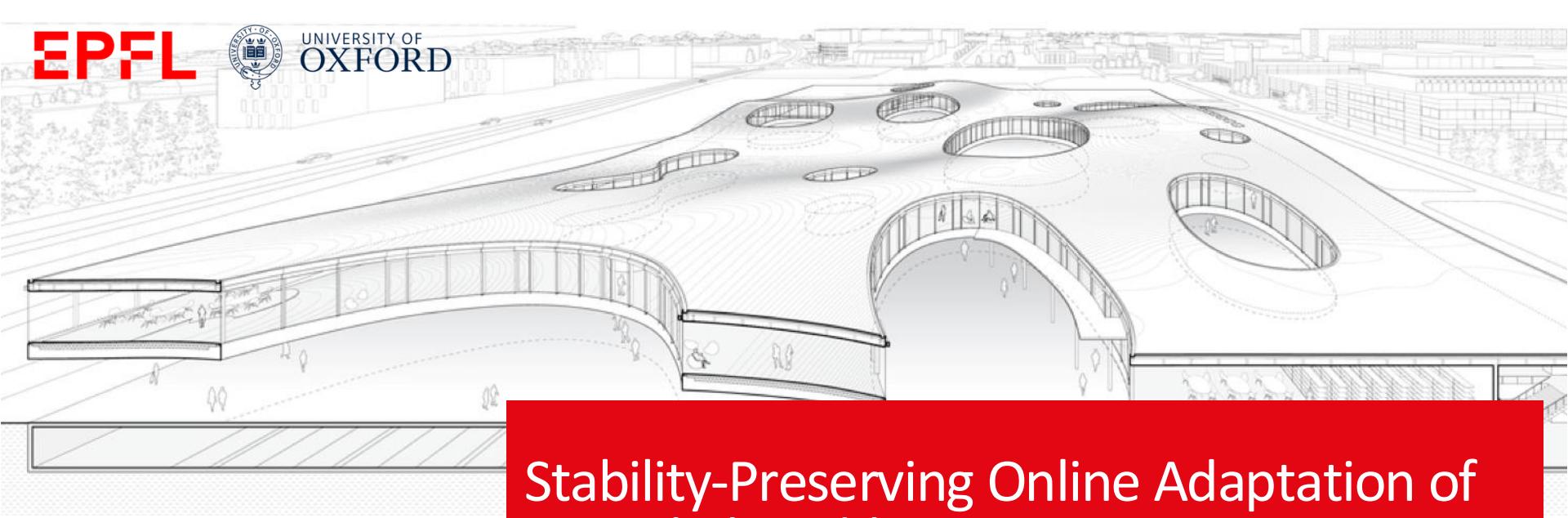


Stability-Preserving Online Adaptation of Neural Closed-loop Maps

Danilo Saccani,
Luca Furieri, Giancarlo Ferrari-Trecate

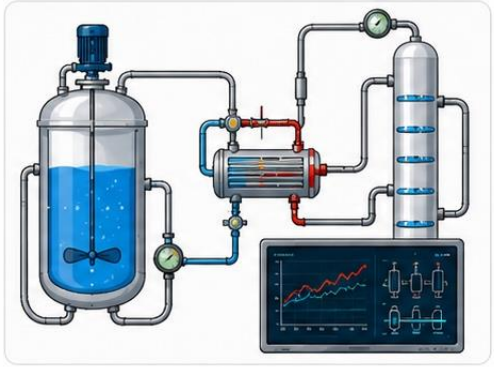
École polytechnique fédérale de Lausanne

Swiss National Science Foundation

NCCR Automation

Motivation

- Time-varying tasks
- Disturbances and changes



- Need for online adaptation
- Fast reactions

- Reliability
- High performance

Stability | **Adaptation** | **Performance**

Can we update a neural controller online without destabilizing the closed loop?

Problem setting

Pre-stabilized system dynamics

$$x_t = f_{t-1}(x_{t-1}, u_{t-1}) + w_t$$
$$f_{-1}(\cdot) = 0, \quad w_0 = x_0$$

Controller

$$u = K(x)$$

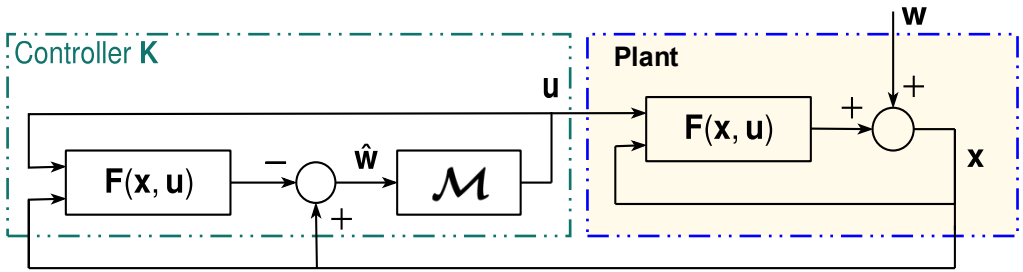
Non-convex objective

$$\min_K \mathbb{E}_{\mathbf{w}} l(\mathbf{x}, \mathbf{u})$$

Closed-loop ℓ_p-stability

$$\mathbf{x} \in \ell_p, \quad \mathbf{u} \in \ell_p$$

➢ Internal Model Control reformulation[1]



All and only stability-preserving controllers can be parametrized through $\mathcal{M} \in \mathcal{L}_p$

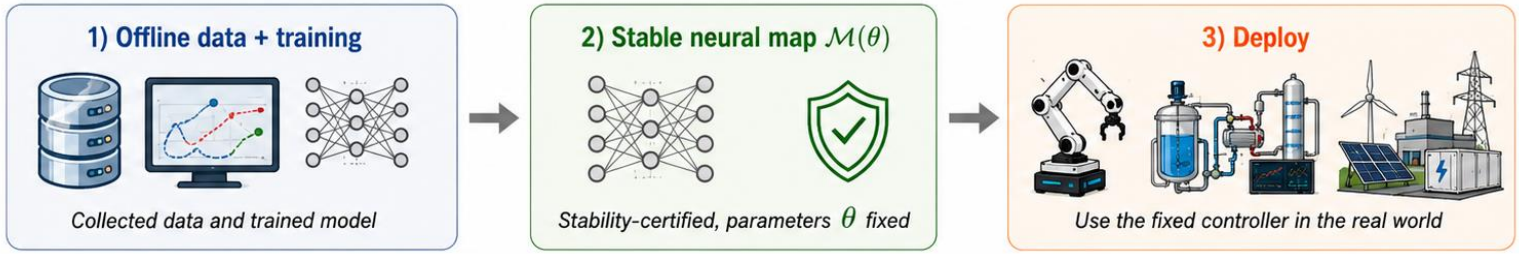
causal and such that: $\mathcal{M}(\hat{\mathbf{w}}) \in \ell_p, \ \forall \hat{\mathbf{w}} \in \ell_p$

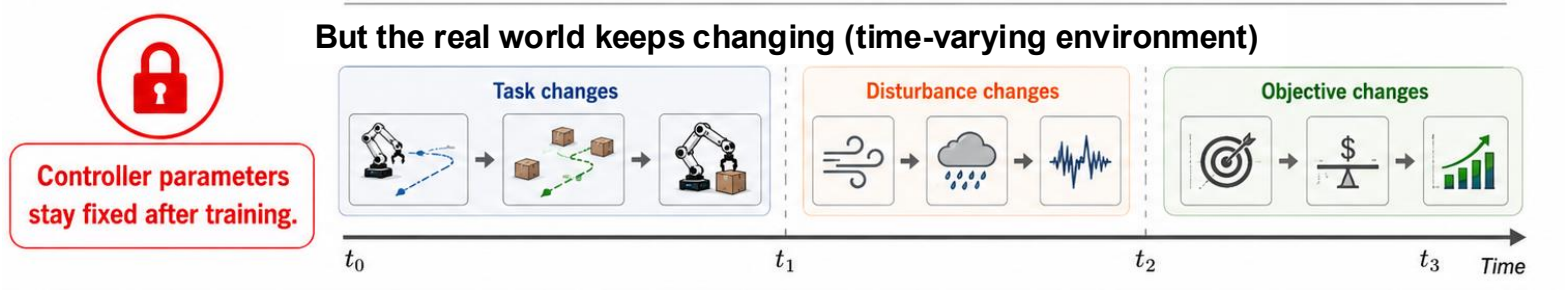
Neural parametrizations (REN/SSM) allow us to guarantee $\mathcal{M}_\theta \in \mathcal{L}_p, \ \forall \theta$
and to enforce a prescribed gain $\gamma(\mathcal{M}_\theta) \leq \bar{\gamma}, \ \forall \theta$

Key idea: search over stable operator rather than generic controllers.

[1] Furieri L. et al. "Learning to boost the performance of stable nonlinear systems." IEEE Open Journal of Control Systems 3 (2024).

Limitation of fixed neural maps

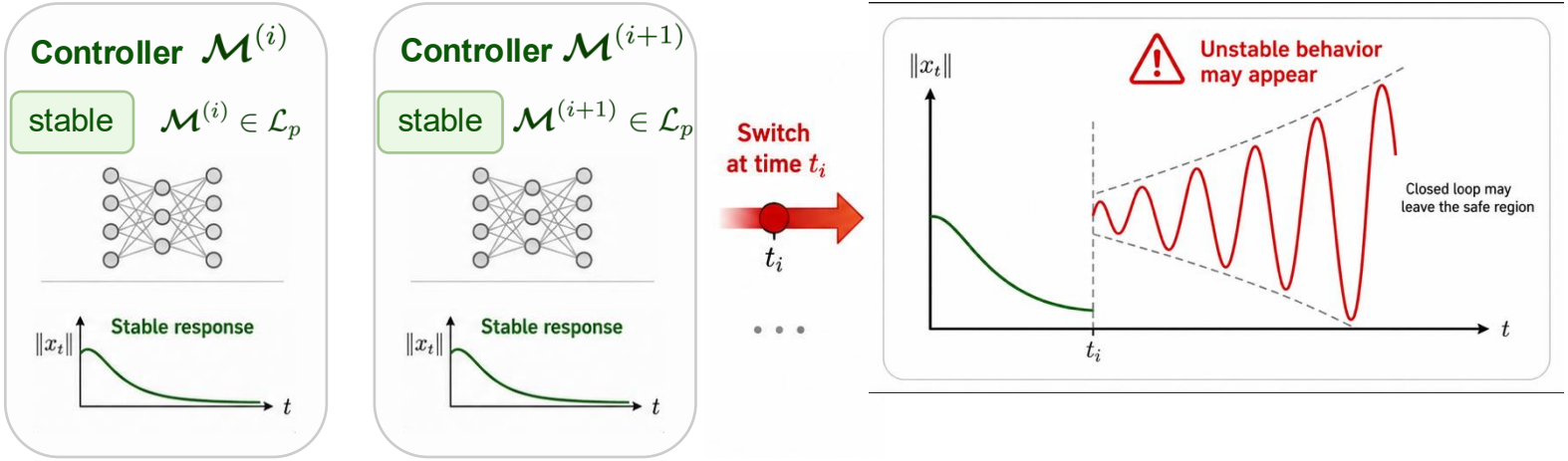




- **Existing stability-preserving neural controllers are time-invariant**
- **Offline training may not cover future operating conditions**
- **We need online adaptation without sacrificing stability**

Good stability certificate exist – but adaptation is missing

Why naive switching fails



Stability of each controller does NOT imply stability under arbitrary switching

- **Each update creates a switched system**
- **Arbitrary updates can break ℓ_p-stability**
- **We need an admissible update rule**

Main challenge: update preserving stability, not just to performe better

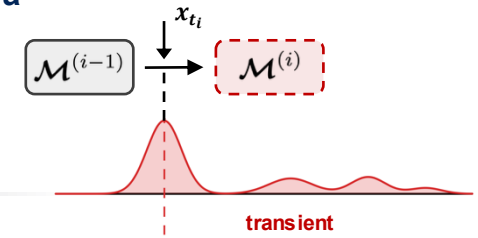
Research question

Under which conditions can we safely replace the current controller with a newly optimized one without losing stability?

1 **Candidate controller is stable by construction**

$$\mathcal{M}^{(i)} \in \mathcal{L}_p$$

2 **But switching injects a transient through x_{t_i}**



3 **Update is safe only if:**

- the state is small enough
- the new controller gain is not too large
- the total adaptation budget remains finite

Gain-budgeted update condition that guarantee the closed loop remains ℓ_p-stable for **any number of updates**

Two **triggering algorithms** to trade off computation and adaptivity

Gain-budgeted stability condition

Theorem (Gain-budgeted stability)

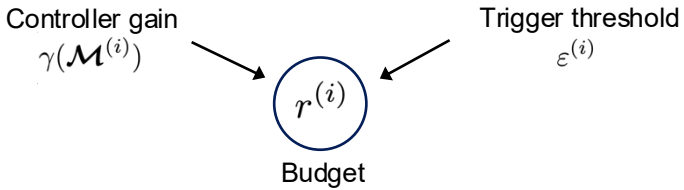
Suppose that at each update $i \geq 1$, the following hold:

1. $\gamma(\mathcal{F})\left(\gamma(\mathcal{M}^{(i)})+1\right)\varepsilon^{(i)} \leq r^{(i)}$
2. $\|x_{t_i}\| \leq \varepsilon^{(i)}$
3. $\mathbf{r} = \{r^{(i)}\}_{i\geq 1} \in \ell_p$

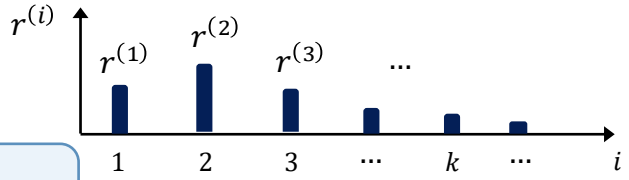
$r^{(i)}$ = adaptation budget at update i

$\Rightarrow$ Then the resulting time-varying closed loop is ℓ_p-stable

r is the main design knob regulating adaptation over time.

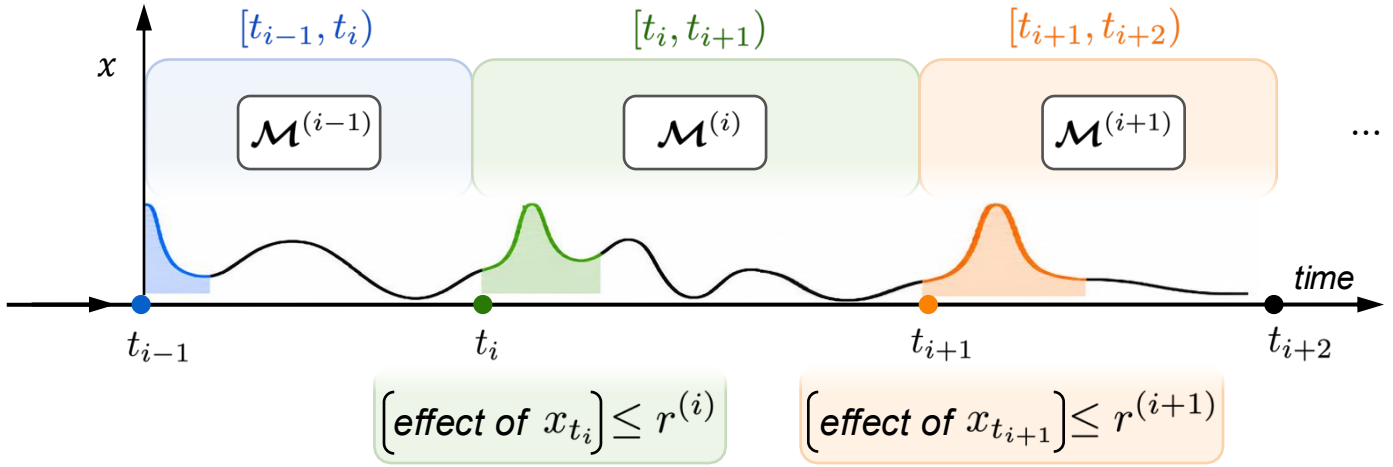


- **Larger** $r^{(i)}$: allows higher gain or looser trigger
- **Smaller** $r^{(i)}$: more conservative update
- $\boldsymbol{r} \in \ell_p$: finite total adaptation budget
- Choose $r^{(i)}$ jointly with $\varepsilon^{(i)}$ and the update times t_i



How does this work?

Why the budget guarantees stability



Each update adds a **bounded** perturbation → $\boldsymbol{r} = \{r^{(i)}\} \in \ell_p$ (total budget finite) → Total perturbation remains finite

Therefore, the time-varying controller is closed loop ℓ_p-stable

Online adaptation framework

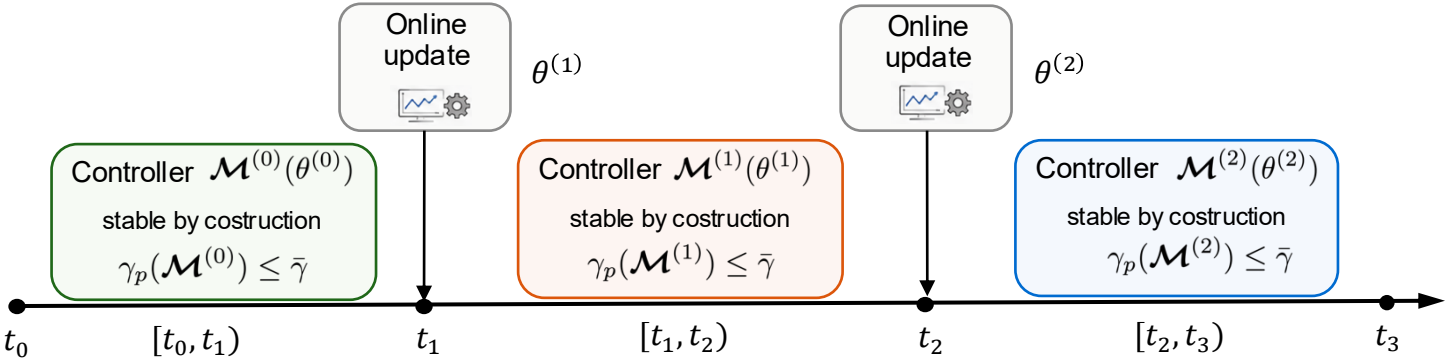
- At time t_i, solve a short-horizon update problem
- Obtain a candidate stable operator $\mathcal{M}^{(i)}$
- Deploy it only if the trigger is satisfied

Composite time-varying operator

$$\widetilde{\mathcal{M}} = \mathcal{M}^{(0)} || \mathcal{M}^{(1)} || \mathcal{M}^{(2)} || \cdots$$

obtained by concatenating stable operators over time

$\widetilde{\mathcal{M}}$ uses $\mathcal{M}^{(i)}$ on $[t_i, t_{i+1})$



- **Train candidate controllers online**
- **Concatenate stable neural operators over time**
- **Update only when admissible**

The controller adapts online, while stability is enforced by the update logic

Algorithm 1: time-scheduled updates

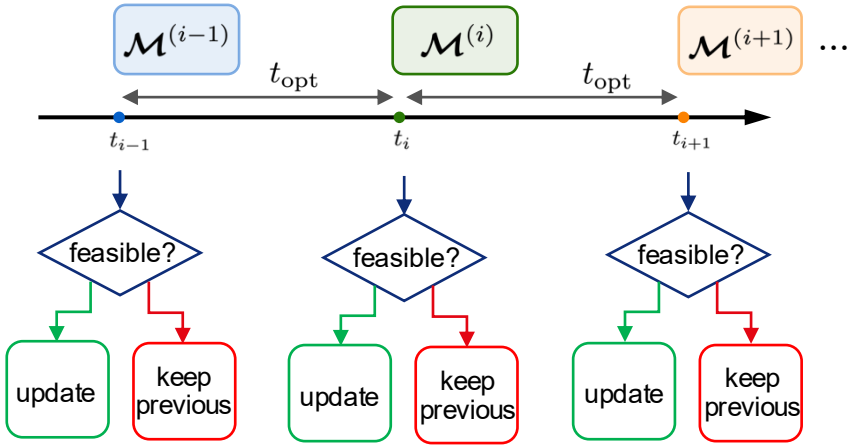
Simple periodic implementation | **Predictable computation load**

Algorithm 1 (time-scheduled updates)

1. Choose an update period t_{opt} and a budget sequence $\mathbf{r} = \{r^{(i)}\}_{i \geq 1} \in \ell_p$
2. At each scheduled time $t_i = it_{\text{opt}}$, measure x_{t_i}
3. Compute $\varepsilon^{(i)}$ so that
$$\gamma(\mathcal{F})\left(\gamma(\mathcal{M}^{(i)}) + 1\right)\varepsilon^{(i)} \leq r^{(i)}$$
4. If $\|x_{t_i}\| \leq \varepsilon^{(i)}$, solve a short-horizon update problem and deploy $\mathcal{M}^{(i)}$; otherwise keep $\mathcal{M}^{(i-1)}$

$\Rightarrow$ Simple implementation with regular computation every t_{opt}

Updates are attempted at fixed periodic times



Next: state-triggered updates reduce unnecessary computations by adapting only when admissible.

Algorithm 2: state-triggered update

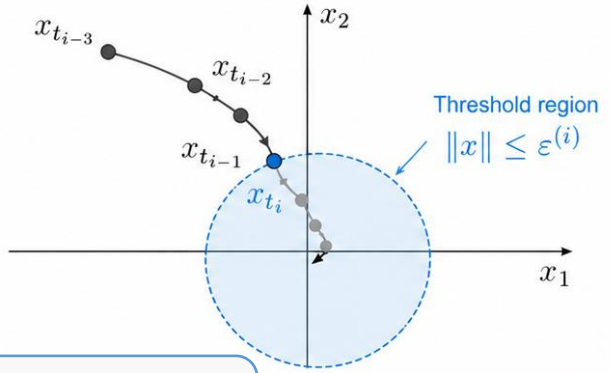
Event-driven implementation | **Adaptive computation load**

Algorithm 2 (state-triggered updates)

1. Choose a budget sequence $\mathbf{r} = \{r^{(i)}\}_{i \geq 1} \in \ell_p$
2. Set thresholds $\varepsilon^{(i)} = \dfrac{r^{(i)}}{\gamma_p(\mathcal{F})(\gamma_p(\mathcal{M}) + 1)}$
3. Monitor the state and wait until $\|x_{t_i}\| \leq \varepsilon^{(i)}$
4. When admissible, solve the update problem and deploy $\mathcal{M}^{(i)}$

$\Rightarrow$ Update only when the state becomes small enough

Updates are triggered only when the state enters the admissible region

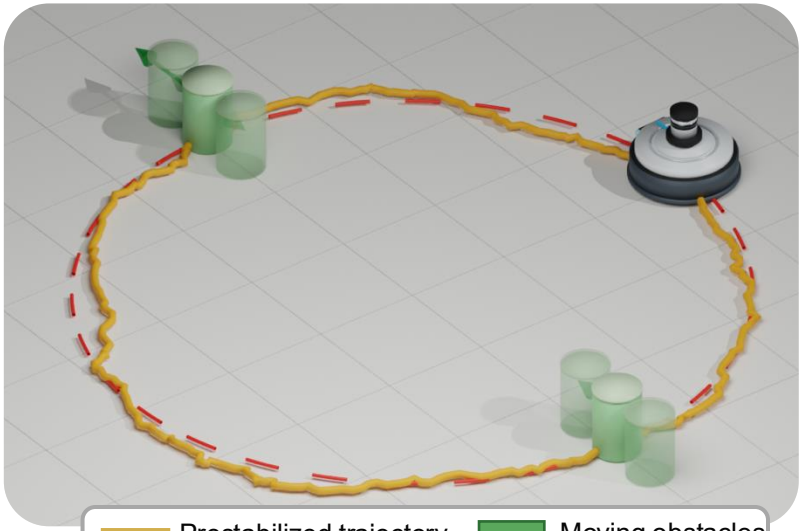


$$\varepsilon^{(i)} = \frac{r^{(i)}}{\gamma_p(\mathcal{F})(\gamma_p(\mathcal{M}) + 1)}$$

Two triggering algorithms to trade off computation and adaptivity

Example: dynamic obstacles

- Point-mass robot with nonlinear friction tracks a circular reference trajectory.
- Moving obstacles create a time-varying collision-avoidance task.
- A pre-stabilizing controller provides the stable baseline behavior



Cost / objective

$$J := \mathbb{E}_{w_{0:T}} \left[\sum_{t=0}^{T} \left(\ell_{\text{traj}}(x_t, u_t) + \ell_{\text{obs}}(x_t, t) \right) \right]$$

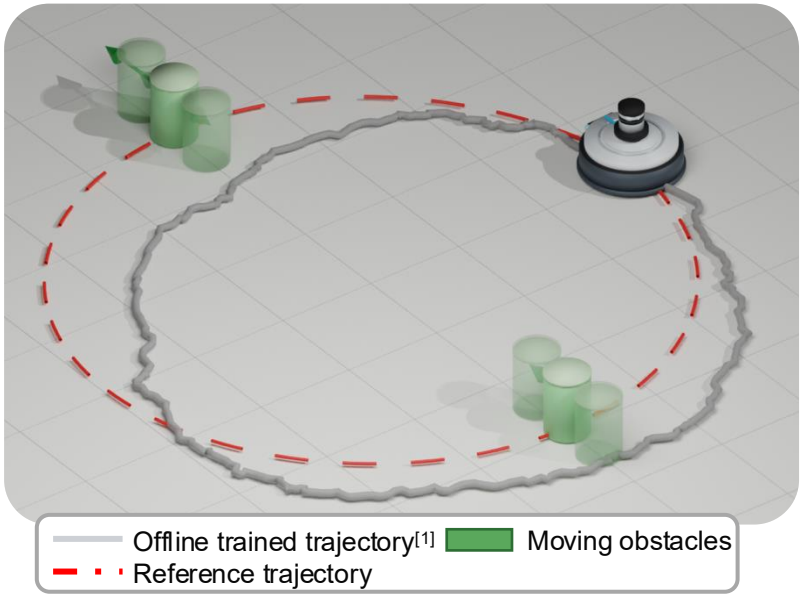
The environment is time-varying, motivating online controller adaptation

Example: dynamic obstacles

- Train a single neural closed-loop map offline[1]

$$\min_{\theta} \mathbb{E}_{w_{0:T}} \left[\sum_{t=0}^{T} \left(\ell_{\text{traj}}(x_t, u_t) + \ell_{\text{obs}}(x_t, t) \right) \right]$$
$$\text{s.t. } x_t = f(x_{t-1}, u_{t-1}) + w_t$$
$$u_t = \mathcal{M}_t^{(0)}(w_t; \theta) \qquad \mathcal{M}^{(0)}$$

- Deploy the same controller for the full task
- Safe and stable but adaptation to new situations is limited



Offline training

Learn one fixed controller $\mathcal{M}^{(0)}$ averaging on representative scenarios

Offline training gives a fixed stabilizing policy, but cannot adapt online

[1] Furieri L. et al. "Learning to boost the performance of stable nonlinear systems." IEEE Open Journal of Control Systems 3 (2024).

Example: dynamic obstacles

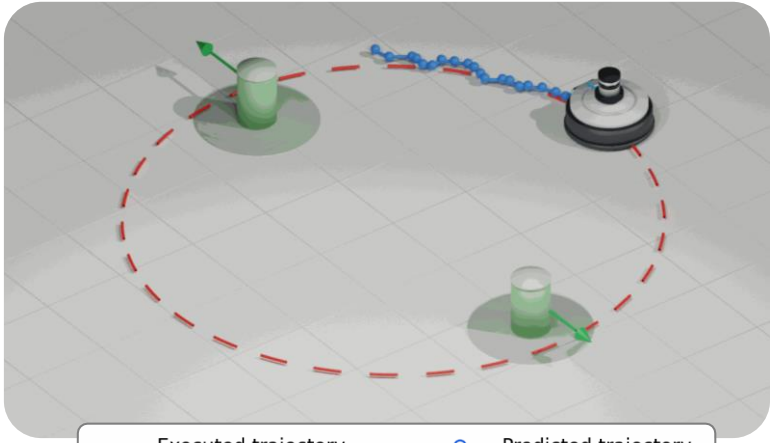
- Update online neural closed-loop map.

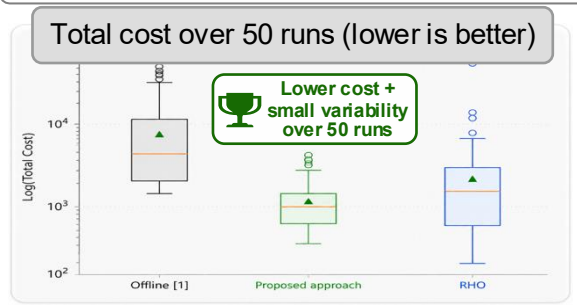
$$\min_{\theta} \frac{1}{S} \sum_{s=1}^{s=S} \left[\sum_{t=0}^{L} \left(\ell_{\text{traj}}(x_t^s, u_t^s) + \ell_{\text{obs}}(x_t^s, t) \right) \right]$$
$$\text{s.t. } x_t^s = f(x_{t-1}^s, u_{t-1}^s) + w_t^s$$
$$u_t^s = \mathcal{M}_t(w_t^s; \theta^{(i)}) \qquad \boldsymbol{\mathcal{M}}^{(i)}$$

- Apply a new controller every t_{opt} when the admissibility condition is satisfied

$$\boldsymbol{\mathcal{M}}^{(0)} \rightarrow \boldsymbol{\mathcal{M}}^{(1)} \rightarrow \boldsymbol{\mathcal{M}}^{(2)}$$

- Improve obstacle avoidance and tracking while preserving stability.





Takeaway: online adaptation improves performance while preserving ℓ_p-stability

Conclusions

Stability

Parametrize controllers through stable operators $\mathcal{M} \in \mathcal{L}_p$

Search over stability-preserving maps

Adaptation

Gain-budgeted condition guarantees ℓ_p-stability

Time-triggered and state-triggered update rules

Performance

Online adaptation improves performance in changing environments

Example: navigation with dynamic obstacles

What this paper shows

- adapt online neural policy
- preserve stability
- improve performance

Next steps

- fast adaptation
- robustness / uncertainty
- hardware and distributed extensions

Adapt online when safe: improve performance without sacrificing ℓ_p-stability

Thanks / Questions?